

Back Off and Give Up

Moshe Zadka – <https://cobordism.com>

Acknowledgement of Country

Hayward (in San Francisco Bay Area)
Ancestral homeland of the Ohlone people

The Secret to Resilient Systems

The Secret to Resilient Systems

Quit

Naive Retries

The problem

Naive Retry: The Classic Mistake

```
import time
import requests

def get_data(url):
    while True:
        try:
            response = requests.get(url, timeout=5)
            return response.json()
        except requests.RequestException:
            time.sleep(1)
```

Thundering Herd

Thundering Herd

Recover

Thundering Herd

Recover
Crash

Retry Storms

Frontend → API Gateway → Payment Service

Retry Storms

Frontend → API Gateway → Payment Service

3

Retry Storms

Frontend → API Gateway → Payment Service

3

×3

Retry Storms

Frontend → API Gateway → Payment Service

3

×3

= 9

Retry Storms

Frontend → API Gateway → Payment Service

3

×3

= 9

$9 \times 100 = 900$

Back-off

Do better!

Exponential

Wait

Exponential

Wait

Proportional to outage

Basic Implementation

```
wait_time = 2 ** attempt  
time.sleep(wait_time)
```

Jitter

Without jitter:

Jitter

Without jitter: Everyone at $T=4s$

Jitter

```
time.sleep(  
    random.uniform(0.9, 1.1)  
    *  
    2**attempt  
)
```

Maximum timeout

```
time.sleep(  
    max(  
        random.uniform(0.9, 1.1)  
        *  
        2**attempt,  
        MAX_TIMEOUT,  
    )  
)
```

What's the maximum?

```
MAX_TIMEOUT = timedelta(seconds=30).seconds()
```

```
MAX_TIMEOUT = timedelta(minutes=10).seconds()
```

A Tale of Two Worlds: Premise

Recipe recommendation site with ML backend

A Tale of Two Worlds: Premise

New model rolled out

A Tale of Two Worlds: Premise

New model rolled out
Overfit on time

A Tale of Two Worlds: Premise

New model rolled out

Overfit on time

Triggers bug

A Tale of Two Worlds: Premise

Fix

A Tale of Two Worlds: Premise

Fix

Rolling roll-back

A Tale of Two Worlds: World 1

Max backoff = 30 second

A Tale of Two Worlds: World 2

Max backoff = 10 minutes

Tenacity

```
@retry(  
    stop=stop_after_attempt(5),  
    wait=wait_exponential(  
        max=600 # 5 minutes!  
    )  
)
```

Giving Up

Tactical retreat

Why Give Up?

Avoid catastrophizing

Circuit Breaker

Three states:

Circuit Breaker

Three states:
Closed

Circuit Breaker

Three states:

Closed

Open

Circuit Breaker

Three states:

Closed

Open

Half-Open

PyBreaker

```
@CircuitBreaker(  
    fail_max=5,  
    reset_timeout=60  
)  
def get_user_data(user_id):  
    return database.query(...)
```

Load Shedding

Priority levels

Optional Features

```
@optional_feature(fallback=[])  
def get_recommendations(user_id):  
    return ml_model.predict(user_id)
```

Putting It Together

Configuration time

Real Configuration

```
retry:  
  max_delay: 300  # 5 minutes!  
  jitter: true
```

```
circuit_breaker:  
  failure_threshold: 5  
  recovery_timeout: 60
```

```
load_shedding:  
  0.7: ["reports"]  
  0.8: ["recommendations"]  
  0.9: ["non_critical"]
```

Monitor Everything

Monitor Everything

Metrics

Monitor Everything

Metrics

Alerts

Key Takeaways

Key Takeaways

- ▶ Exponential backoff with jitter

Key Takeaways

- ▶ Exponential backoff with jitter
- ▶ Max backoff: 5-10 minutes

Key Takeaways

- ▶ Exponential backoff with jitter
- ▶ Max backoff: 5-10 minutes
- ▶ Give up

Key Takeaways

- ▶ Exponential backoff with jitter
- ▶ Max backoff: 5-10 minutes
- ▶ Give up
- ▶ Prioritize

Resources

Libraries:

- ▶ `tenacity`
- ▶ `pybreaker`

Questions?

Moshe Zadka

<https://cobordism.com>